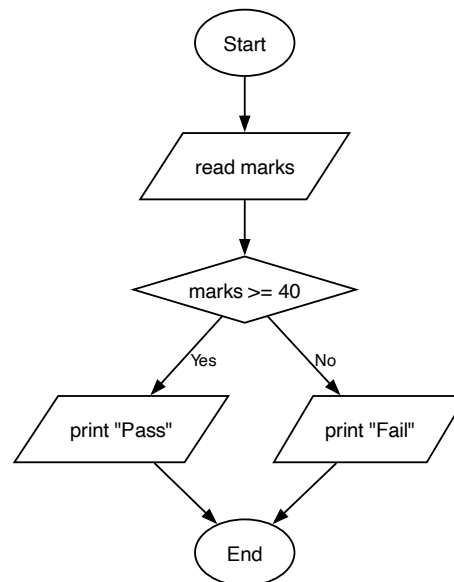


*Answer key***Problem 1.**

A student passes if they score 40 marks or more. The program reads a student's marks and prints "Pass" or "Fail".



input	should print	actually prints
marks = 72	Pass	Pass
marks = 40	Pass	Fail
marks = 12	Fail	Fail

The mistake: the box `marks > 40` should be `marks >= 40`.

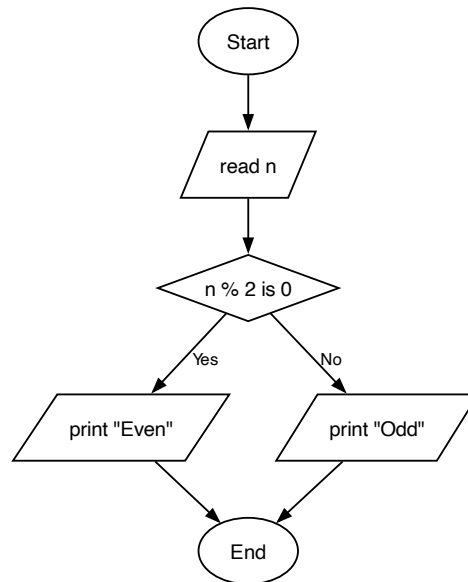
40 marks must pass, so the test has to include 40 itself: "more than 40" leaves the boundary value out. Only the input 40 shows the bug — 72 and 12 look fine, which is why boundary values are always worth testing.

(The flowchart above is the corrected one.)

Problem 2.

The program reads a number and prints “Even” if the number is even, and “Odd” if it is odd.

Reminder: $n \% 2$ is the remainder when n is divided by 2, so $n \% 2$ is 0 exactly when n is even.



input	should print	actually prints
$n = 6$	Even	Odd
$n = 7$	Odd	Even

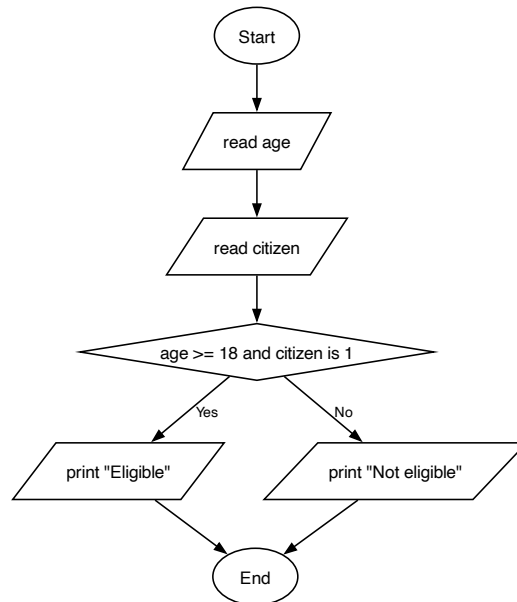
The mistake: the Yes and No arms of the decision $n \% 2$ is 0 are the wrong way round: Yes should lead to print "Even" and No to print "Odd".

The condition is right but the two arms are the wrong way round: the Yes arm (remainder 0) is the even case, so it must print “Even”. Every input comes out wrong — the answers are simply swapped.

(The flowchart above is the corrected one.)

Problem 3.

A person may vote only if they are 18 or older AND they are a citizen. The program reads the person's age and citizen (1 means yes, 0 means no) and prints "Eligible" or "Not eligible".



input	should print	actually prints
age = 20, citizen = 1	Eligible	Eligible
age = 15, citizen = 1	Not eligible	Eligible
age = 30, citizen = 0	Not eligible	Eligible

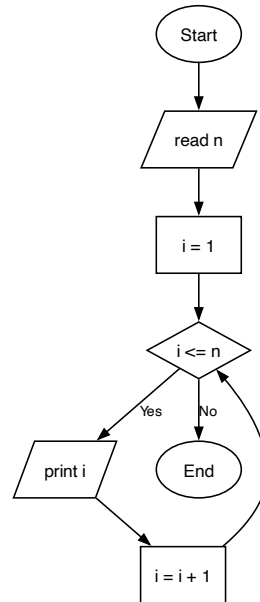
The mistake: the box `age >= 18 or citizen is 1` should be `age >= 18 and citizen is 1`.

Both conditions must hold, so they are joined with “and”. With “or”, meeting just one of them is enough — a 15-year-old citizen and a 30-year-old non-citizen both slip through.

(The flowchart above is the corrected one.)

Problem 4.

The program reads a number n and prints every number from 1 up to n — so for $n = 5$ it prints 1, 2, 3, 4, 5.



input	should print	actually prints
$n = 5$	1, 2, 3, 4, 5	1, 2, 3, 4
$n = 1$	1	nothing

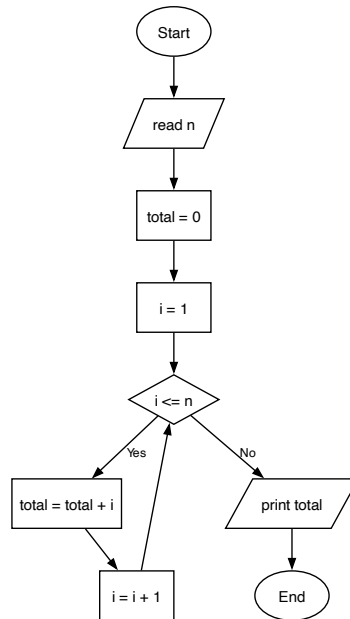
The mistake: the box $i < n$ should be $i \leq n$.

The loop must keep going while i is still 1 to n , and that includes $i = n$ itself. Testing $i < n$ stops one step too early, so the last number is never printed (and for $n = 1$ nothing is printed at all).

(The flowchart above is the corrected one.)

Problem 5.

The program reads a number n and prints the total of all the numbers from 1 to n — so for $n = 5$ it prints $1+2+3+4+5 = 15$.



input	should print	actually prints
$n = 5$	15	16
$n = 3$	6	7

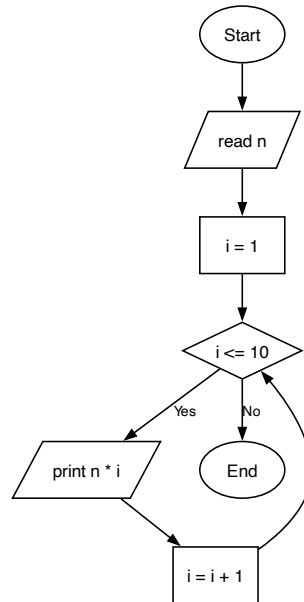
The mistake: the box `total = 1` should be `total = 0`.

Before the loop, nothing has been added yet, so the running total must start at 0. Starting it at 1 adds a stray 1 that is not one of the numbers, so every answer is 1 too big.

(The flowchart above is the corrected one.)

Problem 6.

The program reads a number n and prints its multiplication table from $n \times 1$ up to $n \times 10$ — so for $n = 3$ it prints 3, 6, 9, ..., 30.



input	should print	actually prints
$n = 3$	3, 6, 9, 12, 15, 18, 21, 24, 27, 30	3, 3, 3, 3, 3, ... <i>(never stops)</i>

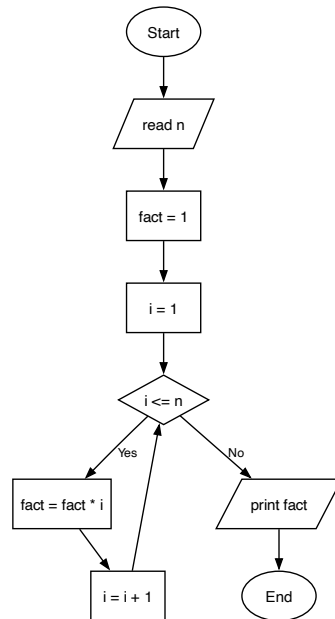
The mistake: the box $i = i + 1$ is missing inside the loop.

Nothing inside the loop changes i , so i stays 1 for ever, the test “ i is 10 or less” is always true, and the loop never ends — it keeps printing $n \times 1$. A counting loop needs its counter to move on every time round.

(The flowchart above is the corrected one.)

Problem 7.

The program reads a number n and prints its factorial — the product $1 \times 2 \times 3 \times \dots \times n$. So for $n = 4$ it prints $1 \times 2 \times 3 \times 4 = 24$.



input	should print	actually prints
n = 4	24	11
n = 5	120	16

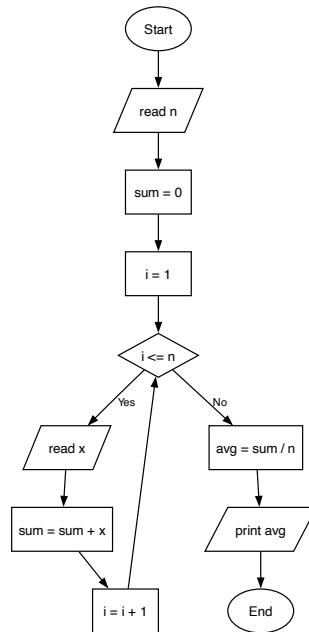
The mistake: the box `fact = fact + i` should be `fact = fact * i`.

A factorial multiplies the numbers together, so the running value must be multiplied by i each time round. Adding instead gives the total of the numbers, not their product.

(The flowchart above is the corrected one.)

Problem 8.

The program reads how many numbers there are (n), then reads that many numbers one by one and prints their average — the total divided by how many there were.



input	should print	actually prints
n = 4, x = 10, 20, 30, 40	25	100
n = 2, x = 7, 9	8	16

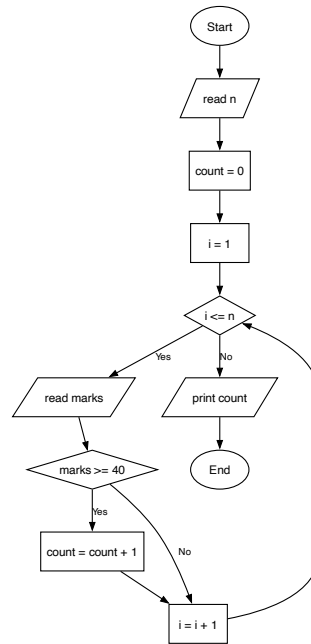
The mistake: the box `print sum` should be `print avg`.

The average is worked out correctly and stored in `avg`, but the program prints `sum`, the running total, instead. The right value is computed and then thrown away.

(The flowchart above is the corrected one.)

Problem 9.

The program reads how many students there are (n), then reads each student's marks and finally prints how many of them passed (40 marks or more). For the marks 55, 32, 80, 40 it should print just one number: 3.



input	should print	actually prints
n = 4, marks = 55, 32, 80, 40	3	1, 1, 2, 3

The mistake: the box `print count` has been drawn inside the loop — it belongs after the loop.

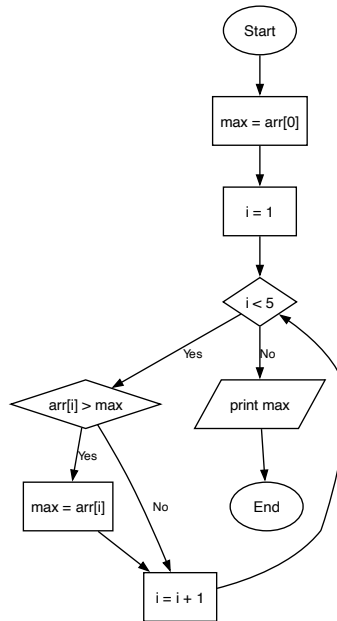
The count is only finished once every student has been read, so the print box belongs after the loop, not inside it. Drawn inside the loop it runs on every turn and prints the count so far each time.

(The flowchart above is the corrected one.)

Problem 10.

The program goes through a list of 5 numbers and prints the largest of them. It starts by assuming the first item is the biggest, then checks every later item and remembers any item that beats the best one so far.

Reminder: the items of a list have positions counted from 0, so the list [3, 9, 2, 12, 7] has 5 items in positions 0 to 4. $\text{arr}[i]$ means the item at position i , so $\text{arr}[0]$ is 3.



input	should print	actually prints
arr = 3, 9, 2, 12, 7	12	2
arr = 8, 1, 6, 4, 5	8	1

The mistake: the box $\text{arr}[i] < \text{max}$ should be $\text{arr}[i] > \text{max}$.

max is meant to hold the biggest value seen so far, so it should be replaced only by an item that is bigger than it. With the comparison the other way round the program keeps the smaller item every time, so it prints the smallest number in the list.

(The flowchart above is the corrected one.)